Cryptology

LAB Assignment 5

Name: Shreyas S Sanil

Roll No: 261100690003

Subject: Cryptology

Shift, Caesar, ROT13, Atbash and Affine Ciphers

Question 1: Shift Cipher — Encryption and Decryption

The shift cipher moves every letter forward (encrypt) or backward (decrypt) by a fixed key, wrapping around the alphabet. Only alphabetic characters are shifted; case and non-letters are preserved.

```
def validate_key(key, modulus=26):
    if not isinstance(key, int):
        raise ValueError("Key must be an integer.")
    return key % modulus
def shift_encrypt(text, key):
    key = validate_key(key)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch
    return result
def shift_decrypt(text, key):
    return shift_encrypt(text, -key)
plaintext = input("Enter plaintext: ")
key = int(input("Enter shift key (0-25): "))
cipher = shift_encrypt(plaintext, key)
print("Encrypted:", cipher)
print("Decrypted:", shift_decrypt(cipher, key))
```

Question 2: Caesar Cipher — Encryption and Decryption

The Caesar cipher is the shift cipher with the key fixed at 3. It is implemented by calling the shift cipher with key = 3, with input validated to contain only letters and spaces.

```
def caesar_encrypt(text):
    if not all(ch.isalpha() or ch.isspace() for ch in text):
        raise ValueError("Plaintext must contain only letters and spaces.")
    return shift_encrypt(text, 3)
def caesar_decrypt(text):
    return shift_encrypt(text, -3)
```

```
plaintext = input("Enter plaintext: ")
cipher = caesar_encrypt(plaintext)
print("Caesar Encrypted:", cipher)
print("Caesar Decrypted:", caesar_decrypt(cipher))
```

Question 3: ROT13 — Encryption and Decryption

ROT13 is the shift cipher with key = 13. Because 13 is half of 26, encryption and decryption are the same operation.

```
def rot13(text):
    return shift_encrypt(text, 13)
plaintext = input("Enter plaintext: ")
encoded = rot13(plaintext)
print("ROT13 Encoded:", encoded)
print("ROT13 Decoded (apply again):", rot13(encoded))
```

Question 4: Atbash Cipher — Encryption and Decryption

The Atbash cipher reverses the alphabet: A maps to Z, B to Y, and so on. It is its own inverse, so the same function encrypts and decrypts.

```
def atbash(text):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr(90 - (ord(ch) - 65))
        elif ch.islower():
            result += chr(122 - (ord(ch) - 97))
        else:
            result += ch
    return result
plaintext = input("Enter plaintext: ")
cipher = atbash(plaintext)
print("Atbash Encrypted:", cipher)
print("Atbash Decrypted:", atbash(cipher))
```

Question 5: Affine Cipher — Encryption and Decryption

The affine cipher encrypts each letter x as E(x) = (a·x + b) mod 26, and decrypts using the modular inverse of a: D(y) = a■¹·(y − b) mod 26. The key 'a' must be coprime with 26, which is validated with gcd before use.

```
import math
```

```
def mod_inverse(a, m=26):
    g = math.gcd(a, m)
    if g != 1:
        raise ValueError(f"'a' = {a} is not coprime with {m}; no inverse exists.")
    return pow(a, -1, m)
def affine_encrypt(text, a, b):
    if math.gcd(a, 26) != 1:
        raise ValueError("Key 'a' must be coprime with 26.")
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((a * (ord(ch) - 65) + b) % 26 + 65)
        elif ch.islower():
            result += chr((a * (ord(ch) - 97) + b) % 26 + 97)
    else:
        result += ch
return result
def affine_decrypt(text, a, b):
    a_inv = mod_inverse(a)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((a_inv * ((ord(ch) - 65) - b)) % 26 + 65)
        elif ch.islower():
            result += chr((a_inv * ((ord(ch) - 97) - b)) % 26 + 97)
        else:
            result += ch
    return result
plaintext = input("Enter plaintext: ")
a = int(input("Enter key 'a' (must be coprime with 26): "))
b = int(input("Enter key 'b': "))
cipher = affine_encrypt(plaintext, a, b)
print("Affine Encrypted:", cipher)
print("Affine Decrypted:", affine_decrypt(cipher, a, b))
```

Sample Run (Affine, a = 5, b = 8, plaintext = "HELLO")

```
Enter plaintext: HELLO
Enter key 'a' (must be coprime with 26): 5
Enter key 'b': 8
Affine Encrypted: RCLLA
Affine Decrypted: HELLO
```

Bonus: Automated Cryptanalysis of the Shift Cipher (Frequency Analysis)

Rather than guessing the shift key by eye, this adds a chi-squared frequency-analysis attack: every one of the 26 possible decryptions is scored against standard English letter frequencies, and the key producing the closest match to real English is chosen automatically, with no plaintext known in advance.

```
ENGLISH_FREQ = {
    'A': 8.2, 'B': 1.5, 'C': 2.8, 'D': 4.3, 'E': 12.7, 'F': 2.2, 'G': 2.0,
    'H': 6.1, 'I': 7.0, 'J': 0.15, 'K': 0.77, 'L': 4.0, 'M': 2.4, 'N': 6.7,
    'O': 7.5, 'P': 1.9, 'Q': 0.095, 'R': 6.0, 'S': 6.3, 'T': 9.1, 'U': 2.8,
    'V': 0.98, 'W': 2.4, 'X': 0.15, 'Y': 2.0, 'Z': 0.074
}
def crack_shift_frequency(ciphertext):
    best_key, best_score = 0, float("inf")
    for key in range(26):
        decrypted = shift_encrypt(ciphertext, -key)
        letters = [c for c in decrypted.upper() if c.isalpha()]
        n = len(letters) or 1
        counts = {c: 0 for c in ENGLISH_FREQ}
        for c in letters:
            counts[c] += 1
        # Chi-squared distance between observed and expected letter frequencies
        score = sum((counts[c] / n * 100 - ENGLISH_FREQ[c]) ** 2 for c in ENGLISH_FREQ)
        if score < best_score:
            best_score, best_key = score, key
    return best_key, shift_encrypt(ciphertext, -best_key)
ciphertext = input("Enter ciphertext (key unknown): ")
key_found, plaintext_found = crack_shift_frequency(ciphertext)
print(f"Recovered key : {key_found}")
print(f"Recovered plaintext: {plaintext_found}")
```

Sample Run

```
Enter ciphertext (key unknown): ESP BFTNV MCZHY QZI UFXAD ZGPC ESP WLKJ OZR
Recovered key : 11
Recovered plaintext: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
```